Immutable Hierarchical Forecast Reconciliation Framework for sktime

Ayush Basu
ayush.basu@studio.unibo.it
GitHub: @ayushhbasu

Contributors

@ayushhbasu

Contents

Code Contribution

PR #9767: ImmutableReconciler Prototype

- OLS-based immutable reconciliation implemented with reduced projection approach
- Basic test infrastructure with 3 validation scenarios
- Currently under review by sktime maintainers (benHeid, felipeangelimvieira, fkiraly, yarnabrina)
- Directly addresses Issue #7586

1 Introduction

Hierarchical forecasting is fundamental to retail, supply chains, energy, and finance, where forecasts must remain coherent across aggregation levels (e.g., national → regional → store-level). Current reconciliation methods in sktime (OLS, WLS, MinT) assume all series are adjustable, preventing real-world scenarios where certain metrics must remain fixed—such as approved budgets, contractual commitments, or regulatory requirements.

This proposal introduces a **comprehensive immutable reconciliation framework** enabling selective constraints within hierarchies. Building on an existing prototype (PR #9767), the solution provides validation, partitioning, and constraint-aware reconciliation while maintaining backward compatibility. The framework integrates seamlessly with **skpro** (probabilistic forecasts) and **prophetverse** (hierarchical Bayesian models).

For preliminary discussions, see:

- Issue #7586: Forecast Reconciliation with immutable series
- PR #9767: ImmutableReconciler Prototype

2 Problem Statement

Current hierarchical reconciliation follows:

$$\hat{y}_{rec} = S\hat{G}\hat{y}$$

Where all series are adjustable. This fails when certain nodes must remain fixed:

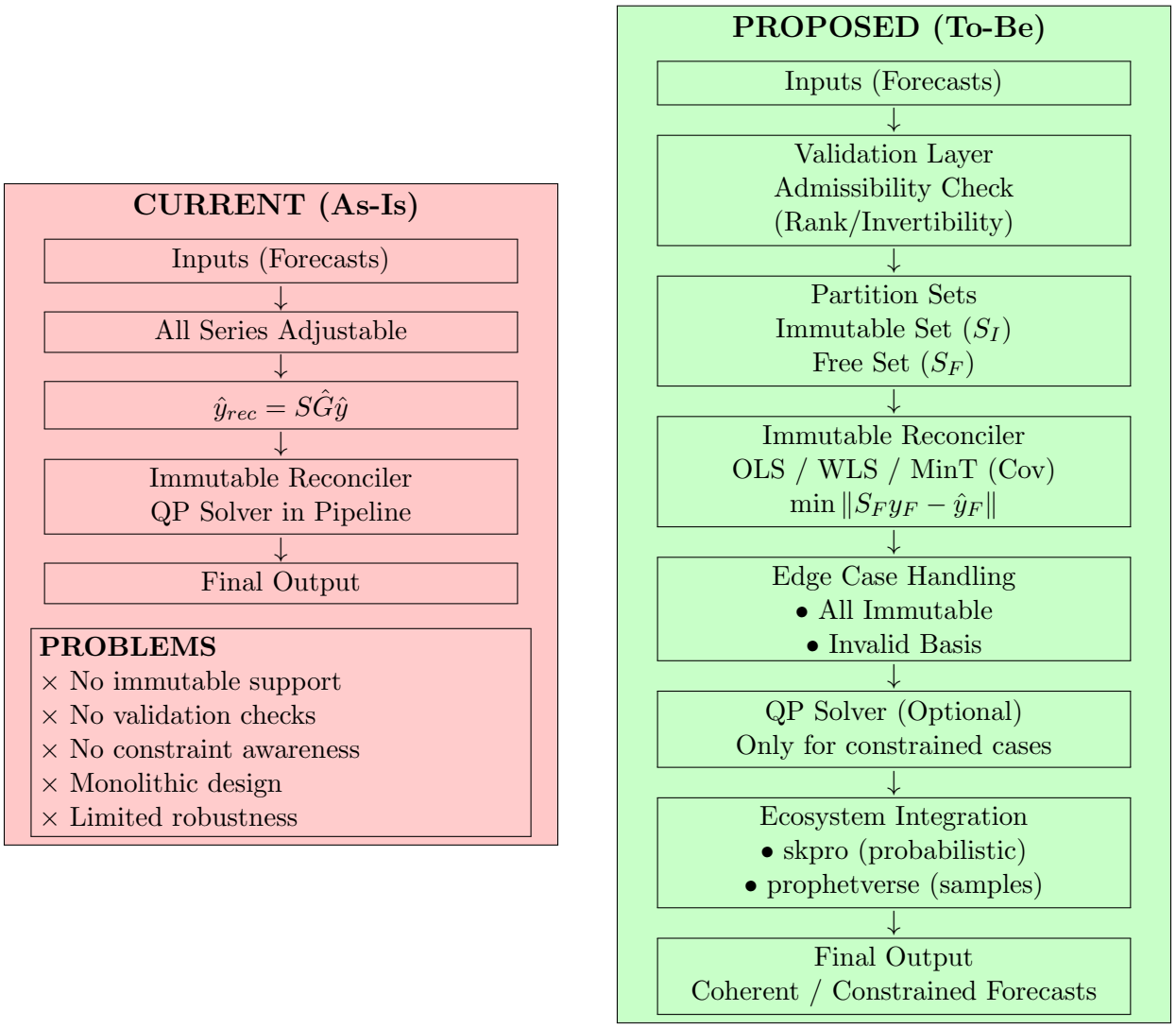
Domain	Immutable Series Examples
Retail	Approved category budgets, contractual commitments
Supply Chain	Fixed production quotas, minimum inventory
Energy	Regulatory caps, renewable targets
Finance	Board-approved forecasts, regulatory disclosures

Key Limitations:

1. No immutability support
2. No validation of immutable sets

3. Monolithic design
4. Always QP (unnecessary overhead)
5. No ecosystem integration (skpro/prophetverse)

3 Architecture Comparison



KEY IMPROVEMENTS

Selective Immutability Modular Architecture Covariance-Aware Methods
Optional QP Solver Robust Edge-Case Ecosystem Integration

4 Description of Proposed Solution

A **modular immutable reconciliation framework** implementing Zhang et al. (2023) [1] with ecosystem support.

4.1 Core Components

Component	Purpose
Validation Layer	Rank/invertibility checks (Zhang et al. Theorem 1)
Partitioning	Identify immutable (S_I) and free (S_F) nodes
Immutable Reconciler	OLS, WLS, MinT with reduced projection
Optional QP	Non-negativity, bounds, linear constraints
Ecosystem Integration	skpro (probabilistic) + prophetverse (hierarchical Bayesian)

4.2 Mathematical Formulation (Zhang et al. 2023)

For immutable indices I and free indices F:

$$y_F^* = \arg\min \|S_F y_F - \hat{y}_F\|$$

$$y_{rec} = \begin{cases} y_I & \text{(immutable)} \\ y_F^* & \text{(reconciled)} \end{cases}$$

4.3 Supported Methods

Method	Use Case
OLS	No covariance information
WLS	Known error covariance
MinT	Estimated covariance (state-of-the-art)

5 Motivation

1. **Community Demand**: Issue #7586 open since 2023
2. **Real-World Need**: Businesses must lock approved forecasts
3. **Research Alignment**: Implements peer-reviewed method [1]
4. **Ecosystem Cohesion**: Works with sktime/skpro/prophetverse
5. **Building on Work**: PR #9767 provides proven foundation

6 Discussion and Comparison of Alternative Solutions

7 Detailed Description of Design and Implementation

7.1 Key Components

1. Validation Layer (Zhang et al. 2023)

Approach	Problem
Post-Reconciliation Masking	Breaks coherence
Penalty-Based	No exact immutability, tuning required
Reduced Projection (Selected)	**Exact, optimal, peer-reviewed**

- Rank condition: $\text{rank}(S_F) = |F|$
- Invertibility: $(S_F^T S_F)$ invertible
- Hierarchical admissibility: Immutable nodes form connected subtree

2. Partitioning

- Identifies immutable nodes + ancestors
- Constructs reduced summing matrix S_F
- Handles multi-index hierarchies

3. Reconciliation Methods (Core OLS Implementation)

Listing 1: Core OLS Reconciliation

```
free_idx = get_free_indices(immutable_idx)
S_free = S[:, free_idx]
y_free = y_pred[free_idx]

# Reduced projection (Zhang et al. Eq. 8)
projection = np.linalg.pinv(S_free)
y_free_rec = S_free @ projection @ y_free

# Reconstruct
y_reconciled = y_pred.copy()
y_reconciled[free_idx] = y_free_rec
```

4. Constraint Handling

- Optional QP solver (cvxopt/osqp)
- Non-negativity, bounds, linear inequalities
- Activated only when needed

5. Ecosystem Integration

- **skpro**: Sample-based reconciliation for Distribution objects; preserves uncertainty quantification; fallback when not installed
- **prophetverse**: Vectorized reconciliation for posterior samples; multi-index hierarchy support; handles correlated uncertainty across levels

6. Edge Cases

- All immutable → return original with warning
- Invalid basis → pseudoinverse with warning
- Rank deficient → regularization
- Empty free set → direct return

7.2 Dependency Strategy

- **skpro** and **prophetverse** as optional dependencies via `extras_require` in `setup.py`
- Lazy import pattern: `_check_soft_dependencies("skpro")`
- Graceful fallback with informative error messages when packages not installed
- Installation: `pip install sktime[forecasting]` includes skpro; `[hierarchical]` extra if maintainers approve

7.3 Testing Strategy

- **Unit tests**: 90%+ coverage for all components (validation, partitioning, methods)
- **Integration tests**: End-to-end testing with sktime forecasting pipeline
- **Property-based tests**: Mathematical invariants (coherence preservation after reconciliation)
- **Benchmark tests**: Performance comparison vs standard reconciliation (¡10% overhead target)
- **Cross-version compatibility**: Python 3.10-3.13

7.4 Risk Mitigation

Risk	Mitigation
MinT covariance estimation complex	Start with sample covariance, add shrinkage iteratively
QP solver performance	Use lightweight osqp, make optional with fallback
API design disagreements	Early design review with mentors (Week 0)
Time constraints	Prioritize core methods first, constraints as stretch
Ecosystem integration issues	Incremental integration: skpro first, then prophetverse

7.5 API Design

Listing 2: ImmutableReconciler API

```
from sktime.forecasting.reconcile import ImmutableReconciler

reconciler = ImmutableReconciler(
    method="mint",                      # "ols", "wls", "mint"
    immutable_series=["national", "region_east"],
    constraints={"non_negative": True},
    preserve_uncertainty=True,          # For probabilistic forecasts
    covariance_estimator="shrinkage"
)

# Works with point, distribution, or samples
y_reconciled = reconciler.reconcile(y_pred, hierarchy)
```

```
# Access components
immutable = reconciler.immutable_forecasts_
reconciled = reconciler.reconciled_forecasts_
```

Backward Compatibility: Existing `Reconciler` code unchanged; new features only when `immutable_series` provided.

8 Timeline (350 Hours)

8.1 Community Bonding

- Finalize design with mentors, review API contracts
- Study sktime/skpro/prophetverse internals
- Setup development environment, pre-commit hooks

8.2 Coding Phase (12 Weeks)

Table 1: Weekly Timeline

Week	Focus	Deliverables
Week 0	Design Finalization	API design approved, development environment ready
Week 1	Validation Layer	`ImmutableValidator` class with rank/invertibility checks; error handling; unit tests
Week 2	Partitioning	Immutable/free detection; hierarchy traversal; `pd.MultiIndex` support; unit tests
Week 3	Core Reconciliation	Extend prototype; reduced projection logic; integration with `BaseReconciler`
Week 4	OLS & WLS	WLS with covariance weighting; benchmarks vs existing methods; documentation
Week 5	MinT Method	Covariance estimation; shrinkage estimators; cross-validation support
Week 6	**Midterm**	Complete core methods; integration tests; **midterm presentation**
Week 7	QP Solver	Constraint handler; non-negativity constraints; osqp integration
Week 8	Constraints	Bound constraints (lower/upper); linear inequalities; validation
Week 9	skpro Integration	Distribution objects; sample-based reconciliation; uncertainty preservation
Week 10	prophetverse Integration	MultiIndex support; posterior samples; vectorized reconciliation
Week 11	Testing	90% coverage; integration tests; property-based tests; benchmarks
Week 12	Documentation & Polish	API docs; user guide; Jupyter examples; final PR

8.3 Final Week

- Final code submission
- Final evaluation
- Demo video

9 Technology Stack

Python 3.10–3.13, NumPy, pandas, sktime, cvxopt/osqp (optional), skpro (optional), prophetverse (optional), pytest, GitHub Actions, Sphinx, Jupyter.

10 Other Commitments

Period	Commitment	Mitigation
May - June 14	Classes (˜20 hrs/week)	˜20 hrs/week for project work, plus weekend coding
June 15-30	Spring semester exams	Reduced coding; documentation & review only
July - August	Summer break	Full availability (40+ hrs/week)

Communication: Daily on GitHub/sktime Slack (CET), weekly mentor check-ins, response within 24 hours.

Organization Preference: sktime (only application)

11 References

1. PR #9767: ImmutableReconciler Prototype
2. Issue #7586: Forecast Reconciliation with immutable series
3. Zhang, B., Kang, Y., Panagiotelis, A., & Li, F. (2023). "Optimal reconciliation with immutable forecasts." *European Journal of Operational Research*, 308(2), 650-660.
4. Wickramasuriya, S. L., et al. (2019). "Optimal forecast reconciliation for hierarchical and grouped time series through trace minimization." *Journal of the American Statistical Association*, 114(526), 804-814.
5. sktime — skpro — prophetverse